Bredden-först-sökning i ett träd

Ardwan Al-Geilani

Våren 2026

Introduktion

I denna uppgift implementeras *Breadth-First-Search*, även kallat för *bredden-först-sökning* och förkortas till BFS. Syftet med uppgiften är att implementera en alternativ strategi för att gå igenom ett binärt träd. Istället för att traversera trädet genom strategin *Depth First*, så används strategin BFS för att besöka noderna nivå för nivå, från vänster till höger.

Träd

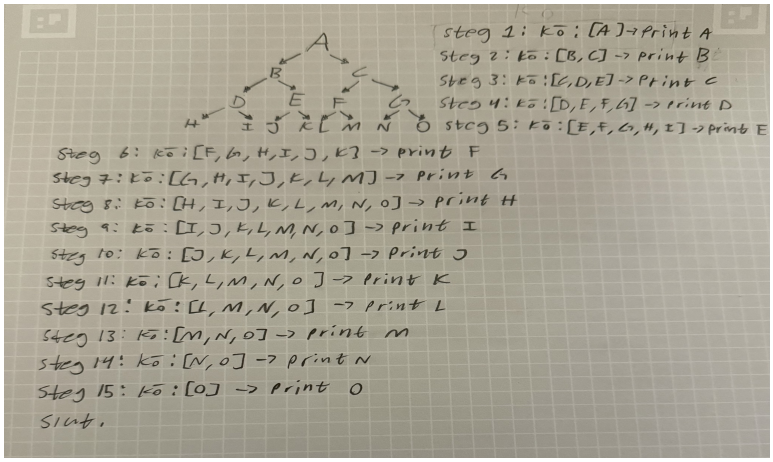
Ett binärt sökträd tar sin början i en så kallad rot. Från denna startpunkt förgrenar sig strukturen nedåt genom att roten pekar vidare mot två underliggande noder: en till vänster och en till höger. Varje enskild nod i trädet bär på ett heltalsvärde (*integer*) och har i sin tur egna pekare till sina respektive vänster- och högernoder. Själva logiken i detta binära träd bygger på en nivåstruktur. För varje given nod förgrenar sig trädet nedåt till nästa nivå, vilket gör det möjligt att traversera strukturen i en specifik ordning oberoende av nodernas faktiska värden.

Implementation

I denna del är implementationen uppdelad i två delar: en kö-struktur för att hantera ordningen på noderna, och en sekvens-struktur som möjliggör *lat*-traversering.

Ritning

Ett binärt träd ritades i fyra nivåer med totalt 15 noder (A–O), se figur 1. I figuren till höger visas köns tillstånd steg för steg, vilket exemplifierar hur kön fungerar som ett minne för att besöka noder nivå för nivå.



Figur 1: Ritning

Kö-strukturen

För att BFS ska fungera krävs en kö som följer FIFO-ordning (First in, First out). Denna version av kön lagrar pekare till trädets noder. När en nod besöks, tas den ut ur kön (dequeue), och dess underliggande noder läggs till sist i kön (enqueue). Detta gör så att alla noder på en nivå bearbetas innan algoritmen går vidare till nästa nivå.

Observera strukturerna nedan. Den första strukturen *kö_nod* används i syfte att varje gång ett tillägg sker i kön, så skapas en ny nod. Den andra strukturen *kö* används i syfte att hålla koll på vart kön börjar och vart den slutar.

```
typedef struct kö_nod { // Definition.
    träd_nod *nod; // Referens till träd-nod.
    struct kö_nod *next; // Referens till nästa element i kön.
} kö_nod; // Alias.

typedef struct kö { // Definition.
    kö_nod *first; // Referens till first.
    kö_nod *last; // Referens till last.
} kö; // Alias.
```

Lat sekvensen

Att göra sekvensen lat innebär att hela trädet inte bearbetas på en gång, istället tar den fram en nod i taget när det behövs. Implementationen består av en *sekvens* som kommer ihåg köns tillstånd mellan varje anrop. Funktionen *next()* utför själva arbetet. Den plockar ut nästa nod ur kön, sparar dess värde och lägger till nodens underliggande noder sist i kön. Genom detta pausas traverseringen efter varje nod och väntar på att användaren ska be om nästa.

Nedan visas implementationen av funktionen *next*, där strukturen av kön används för att hålla ordning på nästa nivå i trädet. Observera att endast centrala delar av koden visas.

```
int next(sekvens *seq) { // Returnerar nästa värde och uppdaterar kön.
    ...
    träd_nod *aktuell = dequeue(seq->q); // Tar ut den främsta träd-noden från kön.
    int val = aktuell->värde; // Sparar undan nodens värde.
    if (aktuell->left != NULL) // Om det finns en vänster-gren.
        enqueue(seq->q, aktuell->left); // Lägg till den sist i kön.
    if (aktuell->right != NULL) // Om det finns en höger-gren.
        enqueue(seq->q, aktuell->right); // Lägg till den sist i kön.
    return val; // Returnerar det aktuella värdet.
}
```

Effektivitet

Effektiviteten i denna BFS-lösning beror på hur många noder trädet har. Eftersom algoritmen måste besöka varje nod exakt en gång för att skriva ut den, blir tidskomplexiteten $O(n)$, där n är antalet noder. Operationerna *enqueue* och *dequeue* utförs på konstant tid, med hjälp av pekarna för *first* och *last*, vilket ger tidskomplexiteten $O(1)$, notera att operationerna är för kön.

Diskussion

En fördel med att använda *bredden-först-sökning* framför *djupet-först-sökning* är dess fullständighet. Om en gren i trädet skulle vara oändligt lång, så kommer en *djupet-först-sökning* att fastna i den grenen för evigt, medan *bredden-först-sökning* garanterat besöker alla noder nivå för nivå, vilket reflekterar en mer fullständig sökning.

Om ett nytt värde läggs till i trädet under pausen för lat-sekvensen, så kommer det värdet att inkluderas i sökningen så länge dess övre-nod ännu inte har tagits ut ur kön. Detta beror på att undre-noder läggs till i kön när dess övre-nod hanteras. Om den övre-noden redan har bearbetats, så kommer det nya värdet inte inkluderas i sökningen eftersom algoritmen blickar framåt. Det nya värdet kommer att finnas i trädet, men den kommer inte att hanteras av kön och kommer därför aldrig att returneras av funktionen *next*.

Att ta bort en nod under pausen innebär en risk eftersom kön lagrar referenser till noder, och om en nod tas bort, så kommer kön att innehålla en pekare till ogiltigt minne, vilket kan leda till att programmet kraschar. Dessutom kommer under-noderna att försvinna, om noden inte är ett löv, vilket innebär att noden är sist i grenen.

Slutsats

Detta experiment har visat hur man kan med hjälp av en kö styra ordningen i en sökning så att det sker nivå för nivå. Genom att implementera en lat-sekvens som ett hjälpmedel så kan algoritmen pausa och fortsätta där den slutade, vilket resulterar i bättre kontroll.

Att använda sig utav en kö är fundamentalt för denna uppgift, eftersom med hälp av kön så kan lat-sekvensen stå i rampljuset. Utan en kö skulle algoritmen glömma bort vilka noder som står på tur så fort funktionen *next* har kört klart. När funktionen anropas återigen så är det kön som gör att lat-sekvensen vet vart den ska fortsätta. Dessutom använder sig en kö utav *FIFO*-ordning, vilket resulterar i en logisk output.